

Arquitetura do Front-end (src)

Este projeto é a camada front-end da aplicação. A partir da pasta `src`, a estrutura é organizada em:

- `pages/`: telas completas da aplicação.
- `components/`: componentes visuais reutilizáveis.
- `contexts/`: estado global de autenticação.
- `hooks/`: lógica de interface aplicada a páginas/componentes.
- `services/`: comunicação com APIs e Firebase.
- `routes/`: rotas protegidas e lógica de navegação.
- `mocks/`: dados fictícios usados em desenvolvimento.

Entrada da aplicação

`src/main.tsx`

- Ponto de partida.
- Importa o CSS global.
- Envolve App com `AuthProvider`.

`src/App.tsx`

- Configura o roteamento com `BrowserRouter`.
- Define rotas públicas e privadas.
- Usa `ProtectedRoute` para proteger páginas que exigem login.
- Exibe o `Navbar` em todas as rotas exceto `/login` e `/register`.
- Faz uma verificação inicial de saúde da API com `getHealth()`.

Autenticação

`src/contexts/AuthContext.tsx`

- Controla o estado de usuário (`user`) e estado de carregamento (`loading`).
- Lê e grava `course` no `localStorage`.
- Implementa `signIn`, `register` e `logout` usando `Firebase`.

`src/contexts/useAuth.ts`

- Hook que entrega o contexto de autenticação a componentes.

`src/routes/ProtectedRoute.tsx`

- Exibe uma tela de carregamento enquanto o estado de autenticação está em progresso.
- Redireciona para `/login` se não houver usuário autenticado.
- Renderiza o conteúdo da rota se o acesso estiver autorizado.

Telas (pages/)

Cada pasta em `src/pages/` representa uma tela:

- `Login/`: tela de login.
- `Register/`: tela de cadastro.
- `Feed/`: feed principal, exibe posts e sugestões.
- `Connections/`: página de conexões.
- `Publish/`: formulário para publicar conteúdo.
- `Profile/`: perfil do usuário com filtros de posts.

Essas telas usam `useAuth`, componentes de UI e serviços para trabalhar.

Componentes (components/)

Componentes reutilizáveis que servem às telas:

- `Navbar/`: menu superior e controle de navegação.
- `NavbarItem/`: item de link para o menu.
- `ProfileCard/`: cartão de dados do usuário.
- `FriendsCard/`: cartão de sugestão/conexão.

- `ProfilePostList/`: lista de posts.
- `ProfilePostSwitcher/`: alternador de visualizações de posts.
- `ActionButton/`: botão de ação com ícone.

Serviços (services/)

`src/services/api.ts`

- Client HTTP genérico usando `fetch`.
- Usa `VITE_API_BASE_URL`.
- Trata JSON, 204 e erros via `ApiError`.

`src/services/posts.ts`

- Funções para posts:
 - `listPosts`
 - `getPost`
 - `createPost`
 - `updatePost`
 - `deletePost`
 - comentários e reposts

`src/services/users.ts`

- Funções para usuários:
 - `listUsers`
 - `getUser`
 - `createUser`
 - `updateUser`
 - `deleteUser`
 - `connectUsers`
 - `listConnections`

`src/services/firebase.ts`

- Inicializa Firebase com variáveis de ambiente.
- Exporta `auth` para uso no contexto.

`src/services/index.ts`

- Reexporta tudo para facilitar importações.

Hooks (hooks/)

`src/hooks/useProfilePosts.ts`

- Hook customizado para obter posts do perfil.
- Ainda usa mock de dados (`mocks/profilePosts.ts`).
- Retorna `posts`, `loading` e `error`.

Mocks (mocks/)

- `profilePosts.ts`: posts fictícios para feed e perfil.
- `connections.ts`: sugestões de conexões e amigos.

Esses arquivos servem para prototipar a interface antes que o backend esteja pronto.

Como a arquitetura se conecta

1. `main.tsx` inicia o app com autenticação global.
2. `App.tsx` define as rotas e decide quando exibir o `Navbar`.
3. `ProtectedRoute` garante que apenas usuários logados acessem páginas privadas.
4. `AuthContext` autentica o usuário via `Firebase`.
5. `pages/` usam componentes, hooks e serviços para renderizar cada tela.
6. `services/api.ts` abstrai a chamada ao backend real.
7. `mocks/` permitem visualização e testes locais sem backend pronto.

Observações finais

- A autenticação do front-end é feita com Firebase Authentication.
- As requisições ao backend devem usar `services/api.ts` com `VITE_API_BASE_URL` configurada.
- O projeto é orientado a telas e componentes, ideal para evolução na parte de UI.
- Os mocks mostram que a interface ainda tem dados de desenvolvimento local.